DefendSec

TECHNICAL WHITE PAPER

Self-hosted host security, explained end to end

Architecture, trust model, deployment, operations, and a complete operator walkthrough

A practical guide for evaluators, administrators, and security operators who want to understand not only how to use DefendSec, but how its components communicate and where its security boundaries live.

Version 1.0 · September 2026

github.com/WASP512/defendsec

About this document

DefendSec is a self-hosted security inventory and host-detection platform. One server receives inventory and security telemetry from agents that you deliberately enroll. The browser console turns that data into fleet status, policy checks, alerts, advisory matches, file-integrity drift, audit history, and operator-approved response workflows.

> **Product boundary.** DefendSec is not mobile-device management. It does not provide remote wipe, device lock, Apple DEP/ABM, Windows CSP, configuration profiles, or an arbitrary remote shell. It is designed to answer "what is on this host, is it healthy, what changed, and what bounded action should an operator take?"

This paper documents the current open-source implementation. Configuration examples use the packaged defaults and a Proxmox container ID of `200`; replace example addresses, hostnames, and identifiers for your environment.

Security notice: commands that expose tokens or enrollment secrets should be run only in a trusted administrative session. Never paste those values into tickets, chat, screenshots, or shell history shared with others.

Contents

1. Executive overview

DefendSec puts a small, explicit trust domain around your fleet. The server and database stay in infrastructure you control. Agents authenticate with client certificates, inventory flows over mTLS gRPC, and active-response commands must carry a valid Ed25519 signature.

What it observes

- OS, hardware, addresses, user, uptime, and online state
- Installed software versions and pending package updates
- Firewall and disk-encryption state
- SHA-256 hashes for watched files
- Security Configuration Assessment results
- Local vulnerability-catalog matches

What operators can do

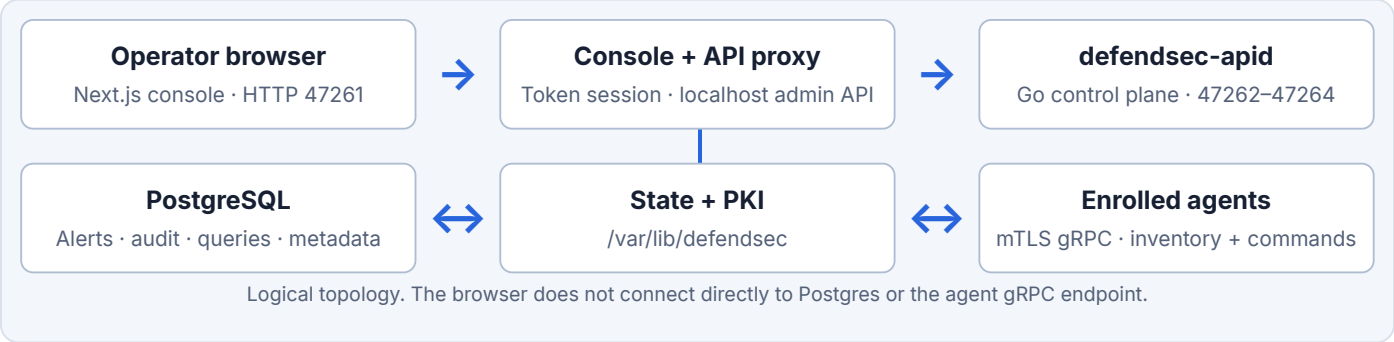
- Triage, acknowledge, resolve, and reopen alerts
- Accept an intentional file-integrity baseline
- Run allowlisted, read-only live queries
- Issue signed containment and response commands
- Revoke a host certificate
- Approve server and agent updates

Design principles

Principle	How the implementation applies it
Self-hosted by default	No vendor account or mandatory cloud service. The console, API, PKI, state, and Postgres run in your VM or LXC.
Explicit enrollment	An agent needs the current enrollment secret to receive a certificate. After enrollment, routine traffic uses mTLS identity.
Bounded response	The agent implements named command types and allowlisted queries/scripts. There is no general-purpose shell command.
Operator approval	Destructive or disruptive actions use confirmations; updates are never silently installed.
Inspectable operation	Commands, acknowledgements, alerts, inventory, and relevant operator activity are retained for review.
Honest scope	The console includes a Scope page and documentation that distinguish security inventory from MDM and enterprise EDR.

2. System architecture

DefendSec separates presentation, control, persistence, and endpoint execution. A typical packaged deployment runs the web console, Go API daemon, and PostgreSQL together in a Debian 12 LXC or Linux VM. Agents run on enrolled hosts.



Logical topology. The browser does not connect directly to Postgres or the agent gRPC endpoint.

Server processes

Component	Responsibility	Default exposure
`defendsec-console`	Next.js standalone application, authentication UI, server-rendered console, agent downloads, and API routes that enforce session roles.	`0.0.0.0:47261`, HTTP
`defendsec-apid`	Enrollment, local CA, gRPC agent hub, inventory/presence, alert and command APIs, signing, Postgres migrations, and persistence.	HTTPS enrollment `47262`; mTLS gRPC `47263`; admin API `127.0.0.1:47264`
PostgreSQL	Durable alerts, alert events/provenance, saved queries, metadata, and related operational records.	`127.0.0.1:5432`
`defendsec-update.path`	Root-owned systemd path unit that triggers the privileged updater when the console writes an approved request file.	Local systemd boundary only

Agent processes

The preferred Go agent, `defendsec-agentd`, collects inventory, maintains the mTLS control stream, watches selected files, evaluates SCA checks, verifies command signatures, executes only recognized command types, and reports acknowledgements. The legacy Python HTTP agent remains useful for inventory-only labs but cannot receive signed commands.

Communication paths

Path	Protocol	Authentication / purpose
Browser → console	HTTP or reverse-proxied HTTPS	Admin or viewer token exchanged for an HTTP-only session cookie.
Console → admin API	HTTP on localhost	Server-side proxy forwards the configured administrative token; port 47264 remains loopback-only.
New agent → enrollment	HTTPS	Enrollment secret authorizes initial certificate issuance on port 47262.
Agent → control plane	Bidirectional gRPC over mTLS	Client certificate identifies the enrolled node on port 47263.

Path	Protocol	Authentication / purpose
Control plane → agent	Protobuf command on the gRPC stream	Ed25519 signature, device binding, issue time, and expiry are verified before execution.
Updater → GitHub Releases	HTTPS	Release metadata and files are downloaded, then validated against `SHA256SUMS`.

3. Technology stack

Web console

Technology	Role
Next.js 16 App Router	Standalone production server, React Server Components, dynamic console pages, API routes, authentication redirects, and response caching controls.
React 19 + TypeScript 5	Typed component model and interactive client workflows.
Tailwind CSS 4	Responsive layout, color tokens, spacing, and print-independent application styling.
Base UI + shadcn-style components	Accessible dialogs, menus, selects, tables, tooltips, sidebar, badges, cards, and form primitives.
TanStack Table	Sortable, filterable, and paginated fleet-style data tables.
cmdk + Lucide	Keyboard command palette and consistent iconography.
`pg`	Server-side Postgres reads needed by selected console routes.

Control plane and endpoint

Technology	Role
Go 1.22+	Produces static-friendly server and agent binaries with predictable memory and process behavior.
gRPC + Protocol Buffers	Typed heartbeat, inventory, hello, bidirectional command, ping, and acknowledgement messages.
Go TLS / X.509	Local CA, client identity, TLS service certificates, certificate fingerprinting, and revocation enforcement.
Ed25519	Signs canonical command content at the server and verifies it at the endpoint.
pgx	Postgres connectivity and transaction handling in the control plane.
fsnotify	Event-driven file-integrity observation in addition to inventory-time hashing.
YAML	Security Configuration Assessment pack definitions.

Packaging and delivery

- **systemd** supervises the console, API daemon, agent, and privileged update workflow.
- **Proxmox** `pct` creates and manages the recommended Debian LXC deployment.
- **Shell installers** provision verified Go/Node toolchains, packages, directories, credentials, builds, and services.
- **GitHub Actions and Releases** turn `v*` tags into amd64/arm64 API and agent binaries, a standalone console archive, installer/updater scripts, `VERSION`, and `SHA256SUMS`.
- **Node's built-in test runner and Go tests** cover release metadata, advisories, cryptography, command bounds, SCA, inventory, PKI, and state behavior.

4. Security and trust model

DefendSec uses several independent controls: operator tokens for the browser, a one-time enrollment secret for certificate issuance, mTLS for ongoing agent identity, Ed25519 for command authenticity, allowlists for endpoint actions, and operating-system boundaries for privileged updates.

Identity lifecycle

1. **Bootstrap.** The installer creates a random admin token, enrollment secret, local CA, server certificate, and command-signing key where needed.
2. **Enroll.** An agent presents the enrollment secret over HTTPS and receives a client certificate and the trust material required for mTLS.
3. **Operate.** The agent connects over mTLS. Its certificate identity is used to associate inventory and command traffic with the node.
4. **Respond.** The API signs a device-bound, time-bounded command. The agent verifies the signature and command type before acting.
5. **Revoke.** An administrator revokes the certificate. The live stream is dropped and the endpoint must clear local identity state and re-enroll.

Browser roles

Capability	Admin token	Viewer token
View fleet, inventory, findings, policies, audit, and update status	Yes	Yes
Enroll hosts or rotate the enrollment secret	Yes	No
Change alert/finding status or accept baselines	Yes	No
Issue signed response commands or revoke certificates	Yes	No
Approve server or agent updates	Yes	No

Command protections

AUTHENTICITY

Commands carry an Ed25519 signature over canonical fields. Unsigned or modified commands fail verification.

SCOPE

The signed content includes the target device identifier, preventing replay to another endpoint.

FRESHNESS

Issue and expiry timestamps bound the window in which a captured command can be accepted.

CAPABILITY

The agent switches on named command types. Live queries and scripts use allowlists; no arbitrary shell is exposed.

> **Network isolation is opt-in.** “Isolate host” always updates DefendSec’s containment state. It changes iptables only when the agent runs as root and `DEFENDSEC_ISOLATE_NET=1` is configured. Test isolation and recovery on a noncritical host before production use.

Network boundary

Expose ports 47261–47263 only to the people and agents that need them. Keep port 47264 and Postgres on loopback. Port 47261 is plain HTTP by default; on any network where token interception is plausible, terminate HTTPS at a trusted reverse proxy and enable secure cookies.

Update privilege boundary

The web console does not execute a root updater directly. An approved admin request is atomically written as `/var/lib/defendsec/update-request.json`. A root-owned systemd path unit notices that one file and starts a constrained oneshot service. The updater validates release identity, repository and URL shape, checksums, archive paths, expected assets, and health before retaining the new version.

5. Data and persistence

Server filesystem

Location	Contents
`/opt/defendsec`	Source/build tree, installed version marker, release trees, and the current standalone-console symlink.
`/var/lib/defendsec`	Admin token, JSON state, enrollment data, PKI, command records, agent download bundle, update status, and backups.
`/etc/defendsec/apid.env`	Control-plane environment including database URL and configured tokens.
`/etc/defendsec/console.env`	Console environment, cookie policy, public URL, installed version, and update repository.
`/etc/defendsec/update.env`	Updater repository, architecture, roots, and state path.
`/usr/local/bin`	Installed API and agent executables.

Agent filesystem

Location	Contents
`/usr/local/bin/defendsec-agentd`	Preferred Go endpoint agent.
`/etc/defendsec/agentd.env`	Server endpoints, TLS server name, state directory, and optional behavior flags.
`/etc/defendsec/enroll-secret`	Enrollment bootstrap secret; protect as root-readable configuration.
`/var/lib/defendsec-agent`	Private node key, issued certificate, CA material, and agent state.

Postgres and migrations

The Go server embeds SQL migrations into the binary and applies them while holding a Postgres advisory transaction lock. Migrations create and evolve alert, provenance, saved-query, and metadata storage. The packaged deployment uses native PostgreSQL by default and verifies the application login during installation. Docker Postgres remains an explicit alternative.

Some inventory and presence data also has a filesystem representation under `/var/lib/defendsec`. Console server loaders merge live mTLS data with stored records and return public projections that omit secrets.

Retention

Data	Default	Environment variable
Resolved alerts	90 days	`DEFENDSEC_ALERT_RETENTION_DAYS`
Live-query results	30 days	`DEFENDSEC_LIVE_QUERY_RETENTION_DAYS`

Pruning occurs when the API daemon starts. Build an external backup and retention schedule around your operational and compliance requirements.

6. Installation

Recommended: Proxmox LXC

Run this on the Proxmox VE host as root. It selects a Debian 12 template, creates an unprivileged container by default, provisions resources, transfers the server installer, and executes it inside the container.

```
curl -fL --progress-bar \
  https://raw.githubusercontent.com/WASP512/defendsec/main/packaging/proxmox/ct/defendsec.sh \
  -o /tmp/defendsec.sh
bash /tmp/defendsec.sh
```

Common overrides:

```
CTID=210 \
CT_HOSTNAME=defendsec \
TEMPLATE_STORAGE=local \
STORAGE=local-lvm \
BRIDGE=vmbr0 \
bash /tmp/defendsec.sh
```

> **Storage types matter.** LXC templates require storage with `vztmpl` content (commonly `local`). Container disks require `rootdir` content (commonly `local-lvm`). Do not send a template to LVM-thin.

Linux VM or existing container

```
curl -fL --progress-bar \
  https://raw.githubusercontent.com/WASP512/defendsec/main/packaging/proxmox/install-server.sh \
  -o /tmp/defendsec-install-server.sh
sudo bash /tmp/defendsec-install-server.sh --advertise-hostname defendsec
```

Supported packaged targets are Debian, Ubuntu, Fedora, and related handled distributions. Native Postgres is the default. The installer preserves an existing database password, admin/viewer tokens, enrollment secret, and PKI on re-run unless replacements are explicitly passed.

Reinstall inside an existing Proxmox container

```
# Run on the Proxmox host; replace 200 with the real CTID.
curl -fL --progress-bar \
  https://raw.githubusercontent.com/WASP512/defendsec/main/packaging/proxmox/install-server.sh \
  -o /tmp/defendsec-install-server.sh
pct push 200 /tmp/defendsec-install-server.sh /tmp/defendsec-install-server.sh
pct exec 200 -- bash /tmp/defendsec-install-server.sh \
  --advertise-hostname defendsec
```

Re-running the server installer refreshes software inside the container. Running `ct/defendsec.sh` creates another container; it is not the upgrade path.

Verify the server

```
pct exec 200 -- systemctl status defendsec-apid defendsec-console --no-pager
pct exec 200 -- curl -fsS https://127.0.0.1:47262/healthz -k
pct exec 200 -- curl -fsS http://127.0.0.1:47261/login >/dev/null
pct exec 200 -- cat /var/lib/defendsec/admin-token.txt
```

A healthy install prints both API and console health-check success before returning.

7. First login and enrollment

1. **Open the console.** Browse to `http://<server-ip>:47261`. The packaged port is HTTP; using HTTPS directly produces an SSL protocol error unless a reverse proxy terminates TLS.
2. **Retrieve the admin token.** On Proxmox, run `pct exec 200 -- cat /var/lib/defendsec/admin-token.txt`. On a VM, run `sudo cat /var/lib/defendsec/admin-token.txt`.
3. **Sign in.** There is no username. Paste the token. The server creates an HTTP-only session cookie and redirects to Fleet.
4. **Open Enroll.** Copy the generated command. It already contains the server URLs, TLS server name, download base, and enrollment secret.
5. **Run it on the endpoint.** Use `sudo` on each Linux amd64 or arm64 host you intend to manage. Do not run the Proxmox container creator on endpoints.
6. **Confirm presence.** Open Hosts. A successful agent normally appears within roughly one minute and is considered online while its latest report is less than two minutes old.

Representative agent install

```
curl -fL --progress-bar \
  "http://SERVER:47261/downloads/install-agent.sh" \
  -o /tmp/install-agent.sh
sudo bash /tmp/install-agent.sh \
  --server-http "https://SERVER:47262" \
  --server-grpc "SERVER:47263" \
  --tls-server-name "defendsec" \
  --enroll-secret "REDACTED" \
  --download-base "http://SERVER:47261/downloads"
```

`--tls-server-name` must match a DNS name or IP address in the server certificate. Use the hostname advertised during server installation.

Offline agent installation

Copy the architecture-specific binary and installer from `/var/lib/defendsec/downloads`, then pass `--binary` to the installer with the same service endpoints and enrollment inputs.

> **Enrollment secret is not a login token.** It authorizes certificate issuance. Treat it as a fleet bootstrap secret, rotate it if exposed, and use certificate revocation for a specific lost or decommissioned endpoint.

8. Console navigation

The responsive sidebar groups destinations by operator intent. Collapse it to icons on wide screens or open it with the menu button on narrow screens. The top bar identifies the active page. The command palette provides keyboard navigation, and theme controls support light, dark, or system appearance.

Group	Destination	Primary question
Monitor	Fleet	What is the overall security posture right now?
	Hosts	Which endpoints are enrolled and what does each report?
	Alerts	What requires triage or response?
	Advisories	Which installed versions match the local vulnerability catalog?
Inventory	Patches	Which updates are pending across hosts?
	Versions	Which software versions are deployed and where?
	Integrity	Which watched files differ from accepted baselines?
	Policies	Which posture checks pass, fail, or remain unknown?
Operations	Audit	What security and operator activity occurred?
	Updates	Is the server or any agent behind the latest stable release?
	Enroll	How do I add a host or rotate enrollment?
	Scope	What does DefendSec intentionally do and not do?

Viewer sessions show a persistent banner, hide Enroll, and remove mutation controls. Signing out clears the session through the logout endpoint.

9. Fleet and Hosts

Fleet dashboard

Fleet is the first operational view after login. Four summary cards answer immediate questions:

- **Hosts online:** last report is inside the two-minute online window.
- **High+ advisories:** open critical or high matches from the local catalog.
- **Pending patches:** total package or OS updates reported by agents.
- **Integrity events:** file-integrity activity plus a summary of failed policy checks.

The host table provides a quick fleet slice. Administrators can enable or disable sample data in environments where that feature is configured, or jump directly to Enroll.

Hosts list

Use Hosts for the complete endpoint roster. Search and table controls support fleet navigation. Status badges separate online/offline hosts, and selecting a row opens the host detail view.

Host detail

Section	What to inspect
Identity and status	Hostname, OS, architecture, online state, mTLS marker, sample marker, and isolation state.
Host facts	Platform, model, serial, CPU, memory, current user, IP addresses, uptime, enrollment time, and last-seen time.
Open alerts	A direct link filtered to this device, or the complete device alert history.
Signed response	Containment, certificate revocation, process response, live queries, agent update, and command history.
Policies	Per-host status and detail for every built-in policy definition.
Advisories	Package/version matches, severity, CVE, and triage state.
Pending patches	Installed-to-available version transitions, or collection status if unsupported/error/unknown.
File integrity	Current SHA-256, accepted baseline, drift paths, and change-event history.
Software versions	Latest package/software inventory reported by the agent.

10. Alerts and Advisories

Alerts: the triage inbox

Alerts combine FIM, Security Configuration Assessment, vulnerability, and response-related findings into an operational queue backed by Postgres. Filter by status, severity, kind, or device. Open an item to examine source context and lifecycle history.

Administrators can:

- **Acknowledge** an alert that is understood but not yet remediated.
- **Resolve** an alert when the condition is fixed or dispositioned.
- **Reopen** an item when a condition returns or needs further work.

Open FIM drift and SCA findings can auto-resolve when later host data demonstrates a return to baseline. Provenance fields preserve the source identity used to correlate recurring observations.

Advisories: catalog matching

Advisories compare reported software names and versions to a local CVE-shaped catalog. Findings indicate that an installed version is below a catalog floor; they are not exploitability guarantees. Use severity and host context to prioritize investigation.

> **Catalog freshness is an operator responsibility.** The repository includes a local catalog. Optional OSV ingest tooling and a systemd timer example exist, but external advisory synchronization is not enabled automatically. Version semantics vary across ecosystems, so confirm high-impact matches against the vendor's own advisory.

11. Patches, Versions, Integrity, and Policies

Patches

Patches aggregates pending updates reported by enrolled agents. Use it to identify hosts with outstanding package changes and to compare current versus available versions. DefendSec reports these gaps; it does not currently orchestrate OS patch installation or reboot windows.

Versions

Versions pivots software inventory across the fleet. Search for a package or product to see which versions exist and which hosts report them. This is useful for incident scoping, end-of-life discovery, and validating a manual rollout.

Integrity

Integrity compares current SHA-256 hashes against an accepted baseline for a deliberately small set of sensitive system paths. A missing file or changed hash is drift. Open the host to review the exact path, current hash, prior hash, and timestamps.

1. Investigate the change on the host.
2. If unauthorized, remediate the file and wait for the agent to report the restored state.
3. If authorized, use **Accept baseline** as an administrator.
4. Review the resulting activity in host history, Alerts, and Audit as applicable.

This is intentionally a focused hash/watch model, not a full enterprise FIM policy engine with broad path tuning and signed policy distribution.

Policies

Policies evaluates seven built-in posture checks for every host. Results are PASS FAIL or UNKNOWN .

Policy	Evaluation
Agent checking in	Inventory received within the last two minutes.
Disk encryption	FileVault, BitLocker, or LUKS reported enabled; unknown if the agent cannot determine it.
Host firewall	Built-in OS firewall reported enabled.
Supported OS	Current rules recognize macOS 14+, Windows 11/Server 2022/2025, and Ubuntu-like version 22.04+.
No high/critical advisories	No open serious finding from the local advisory catalog.
No pending patches	Patch inventory succeeded and returned no updates.
Watched files unchanged	Current hashes match the accepted baseline.

An unknown result means DefendSec lacks sufficient telemetry; it should not be interpreted as a pass.

12. Signed host response

Response controls appear on a host only when it has the preferred mTLS Go agent. The Python inventory agent has no command channel. Viewer sessions may inspect command history but cannot submit actions.

Containment and certificate controls

Action	Behavior	Operator caution
Isolate host	Sends a signed isolate command and records isolated state. Optional network enforcement uses an agent-managed iptables path.	Confirm on a test host; ensure you retain a recovery path.
Release isolation	Sends the signed inverse command and removes agent-managed network isolation/state.	Verify connectivity and agent check-in afterward.
Revoke certificate	Revokes the node identity and drops its live gRPC connection.	The host cannot reconnect until local agent state is cleared and it re-enrolls.

Process response

Process response sends `SIGTERM` to processes whose exact Linux `/proc/*/comm` name matches the submitted value. Names are syntax-limited, and DefendSec refuses to signal a protected set that includes its own processes, systemd/init, SSH, and the console. It is intentionally narrower than passing a shell command. Review the acknowledgement and command history to confirm acceptance and result.

Live query

Choose one of the built-in read-only snapshots:

`processes · listening_ports · users · logged_in_users · crontab · systemd_units · mounts · os_info`

Results render as a compact table when tabular. Up to 50 rows are shown inline before an overflow summary. Save a named preset to make a commonly used query easy to repeat; presets still reference only an allowlisted query identifier.

Agent update

Stable release metadata pre-fills version, architecture-specific URL, and SHA-256. The command instructs the agent to download the candidate, verify its digest, preserve the old binary as a backup, atomically replace the executable, and allow systemd to restart it. The fleet Updates page is the preferred way to target all eligible outdated online agents.

Additional control-plane capabilities

The signed-command API and Go agent also implement two bounded Linux actions that do not currently have general host-page controls:

Command	Restriction	Effect
`run_script`	Only `collect_journal_tail` or `flush_dns`; unknown identifiers are rejected before signing and again at the agent.	Collects the last 50 journal lines, or asks `resolvectl` / `systemd-resolve` to flush DNS caches.
`quarantine_path`	Only a file under the agent's FIM watch paths or its dedicated staging prefix; directories and other paths are refused.	Moves the file into a private agent quarantine directory and removes all file permissions.

Command history

Each entry records type, queued/sent/acknowledged state, whether the endpoint accepted it, result text, payload context, and updated time. Treat “sent” as transport state, not proof that the desired endpoint effect occurred; read the acknowledgement and validate the host.

13. Audit, Updates, Enroll, and Scope

Audit

Audit provides a chronological record of security-relevant and operator activity available to the console. Use it to reconstruct alert transitions, response activity, enrollment or certificate events, and other recorded changes. Pair application audit with systemd and reverse-proxy logs for a complete operational timeline.

Updates

The page compares the installed server version with the latest stable GitHub release and shows enrolled agent versions by architecture and connectivity.

- **Install and restart:** admin confirmation starts a verified, health-gated server update.
- **Update outdated agents:** queues signed, architecture-matched commands for eligible online agents.
- **Status:** the console polls downloading, backing-up, installing, completed, or failed state.
- **Viewer behavior:** update availability and progress are visible, but action buttons are disabled.

If an installation predates the update helper, re-run `install-server.sh` once. This installs `/usr/local/sbin/defendsec-update` and the systemd path unit. Future stable releases no longer require an on-server source build.

Enroll

Enroll generates the preferred Go-agent install command and presents values matching the configured public console URL and advertised TLS identity. Admins can rotate the enrollment secret when needed. Existing certificate-authenticated agents continue operating; rotation prevents reuse of the old bootstrap secret for new issuance.

Scope

Scope explains the product boundary: self-hosted inventory, patch visibility, local advisory matching, and focused file integrity are practical without a vendor control plane; genuine MDM functions rely on Apple, Microsoft, or Google ecosystems and are intentionally excluded.

14. Operator workflows

Daily fleet review

1. Open Fleet and review offline hosts, high+ advisories, pending patches, and integrity-event counts.
2. Open Alerts filtered to **open**; prioritize critical/high and unexpected FIM activity.
3. Open affected hosts to correlate identity, last seen, policy status, pending changes, and command history.
4. Acknowledge items under investigation; resolve only with evidence that the condition is remediated or accepted.
5. Review Updates and apply stable releases during an approved change window.

Investigate suspicious file drift

1. Open the FIM alert, note path, device, first/last seen, and prior/current hashes.
2. Open the host and inspect recent host facts and related integrity events.
3. Use allowlisted queries such as `processes`, `logged_in_users`, and `listening_ports` for volatile context.
4. If risk warrants and containment is tested, confirm **Isolate host**.
5. Remediate out of band. Release isolation only after validation.
6. Accept the new baseline only when the changed file is authorized and known-good.

Revoke and re-enroll a host

1. On host detail, confirm **Revoke certificate**.
2. On the endpoint: `sudo systemctl stop defendsec-agentd`.
3. Remove node identity state: `sudo rm -rf /var/lib/defendsec-agent`.
4. Copy the current command from Enroll and run it on the endpoint.
5. Verify the new certificate-backed host reconnects and inventory returns.

Plan a patch cycle

1. Use Patches to identify hosts and packages with updates.
2. Use Advisories to raise priority where a pending package corresponds to a serious local match.
3. Patch with your normal operating-system tooling and change controls.
4. Wait for the next agent inventory and confirm Patches, Versions, Policies, and Alerts reflect the new state.

15. Production operations

Health and logs

```
# Server or inside the container
systemctl status defendsec-apid defendsec-console
journalctl -u defendsec-apid -u defendsec-console -f
curl -fsS http://127.0.0.1:47261/login >/dev/null && echo console-ok

# Enrolled endpoint
systemctl status defendsec-agentd
journalctl -u defendsec-agentd -f
```

Backup

```
sudo bash -c 'set -a; . /etc/defendsec/apid.env; \
  DEFENDSEC_DATA_DIR=/var/lib/defendsec \
  /opt/defendsec/scripts/backup.sh'
```

The archive contains application state, PKI, admin token, and an optional Postgres dump. Store it encrypted with access controls appropriate for a credential and CA backup.

Restore

```
sudo systemctl stop defendsec-console defendsec-apid
sudo bash -c 'set -a; . /etc/defendsec/apid.env; \
  DEFENDSEC_DATA_DIR=/var/lib/defendsec \
  /opt/defendsec/scripts/restore.sh /path/to/backup.tar.gz'
sudo systemctl start defendsec-apid defendsec-console
```

Confirm hosts reappear and existing agents reconnect with their certificates. Practice restore on a nonproduction instance; a backup that has never been restored is an untested assumption.

HTTPS reverse proxy

1. Terminate TLS on a trusted Caddy, nginx, HAProxy, or equivalent frontend.
2. Forward browser traffic to `http://127.0.0.1:47261`.
3. Forward `X-Forwarded-Host` and `X-Forwarded-Proto`.
4. Set `DEFENDSEC_COOKIE_SECURE=true` and `DEFENDSEC_PUBLIC_CONSOLE_URL=https://defendsec.example.com` in `/etc/defendsec/console.env`.
5. Restart `defendsec-console`. Keep agent ports 47262/47263 reachable or proxy them separately.

Viewer access

Create a long random token and pass it on an installer re-run with `--viewer-token`, or set the same `DEFENDSEC_VIEWER_TOKEN` in both API and console environments and restart both units. Viewer tokens are suitable for read-only monitoring displays; they are still secrets.

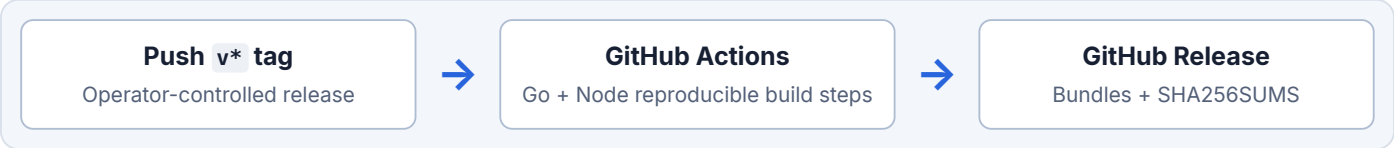
Operational checklist

- ✓ Restrict ports 47261–47263 to administrative and agent networks.
- ✓ Keep 47264 and 5432 loopback-only.
- ✓ Use HTTPS and secure cookies when the browser path is not a trusted isolated LAN.

- ✓ Back up state, PKI, and Postgres on a tested schedule.
- ✓ Monitor systemd units and disk space.
- ✓ Rotate exposed tokens or enrollment secrets promptly.
- ✓ Use viewer credentials where mutation is unnecessary.
- ✓ Test isolation, release, restore, and update rollback before relying on them.

16. Release and update mechanics

Release production



The build produces API and agent binaries for Linux amd64/arm64, a Next.js standalone console archive, agent/server installer scripts, updater script, canonical `VERSION`, and a checksum manifest. Agent versions are injected at link time.

Server update sequence

1. The console reads the installed version and latest stable GitHub release, requiring all expected assets.
2. An administrator confirms **Install and restart**. The console writes a narrow request file.
3. The systemd path unit starts the updater as root and serializes work with a lock.
4. The updater validates repository, tag, version, release identity, asset URLs, checksums, and safe archive paths.
5. It creates a private pre-upgrade archive of state/PKI and runs `pg_dump` when a database URL is available.
6. It stages the release, preserves the API binary as `.bak`, atomically changes the console symlink, refreshes agent downloads, and restarts services.
7. It waits for both API and console health. Success marks the update complete; failure restores previous binaries/symlink and restarts the old version.

> **Rollback boundary.** Binary and console rollback is automatic after failed health checks. The pre-upgrade archive and Postgres dump protect data, but a database restore remains an explicit operator operation. Review migrations and backup status before major-version changes.

Agent rollout sequence

Updates maps release artifacts to endpoint architecture, then marks only outdated, online, supported agents as eligible. After administrator confirmation, the console submits signed update commands. Each agent independently verifies SHA-256 before replacement.

17. Troubleshooting

Symptom	Likely cause	Action
Console TLS/protocol error on 47261	HTTPS used against the default HTTP listener	Use `http://SERVER:47261`, or configure a reverse proxy.
Console or API unavailable	Install/build failure or service crash	Inspect `systemctl status` and `journalctl` for both units.
Agent TLS verification fails	`--tls-server-name` does not match the certificate SAN	Use the advertised server hostname/IP. Regenerate PKI only with a planned fleet re-enrollment.
Agent does not appear	Wrong endpoints, secret, firewall, or service state	Check agent logs, ports 47262/47263, time synchronization, and the current enrollment command.
Agent downloads return 404	Bundle not generated or public URL wrong	Check `/var/lib/defendsec/downloads` and `DEFENDSEC_PUBLIC_CONSOLE_URL`.
Proxmox template download fails on `local-lvm`	LVM-thin cannot store `vztmpl`	Set `TEMPLATE_STORAGE=local` and keep CT disks on `local-lvm`.
Git reports dubious ownership in `/opt/defendsec`	Root installer re-runs against the `defendsec`-owned tree	Use the current installer; it applies a process-scoped safe-directory setting for that exact path.
Installer prints `Using Go VERSION ()`	Old installer lost the toolchain path in a subshell	Download the latest installer from `main` and re-run it.
Update fails	Asset/checksum/backup/health failure	Inspect `defendsec-update.service`, its journal, and `/var/lib/defendsec/update-status.json`.
Forgot admin token	Token is intentionally not printed at completion	`pct exec <CTID> -- cat /var/lib/defendsec/admin-token.txt`.

Update diagnostics

```
systemctl status defendsec-update.service
journalctl -u defendsec-update.service --no-pager -n 100
cat /var/lib/defendsec/update-status.json
```

Do not destroy evidence prematurely

Before purging state, capture service logs, update status, recent audit events, relevant Postgres records, and a protected backup. Reinstalling over the same data is generally nondestructive; destroying an LXC or using uninstall with `--purge-data` is not.

18. Limitations and design tradeoffs

Area	Current boundary	Operational implication
MDM	No wipe, lock, profiles, DEP/ABM, Windows CSP, or Android Enterprise.	Use a real MDM alongside DefendSec when those controls are required.
Vulnerability intelligence	Local catalog; optional OSV ingest is not enabled by default. Cross-ecosystem version comparison has limits.	Maintain catalog freshness and confirm consequential matches with upstream vendors.
Patching	Reports pending updates; does not install OS/package patches or orchestrate reboot windows.	Use existing configuration/patch management for remediation.
File integrity	Focused watched paths and accepted hashes, not a highly tuned enterprise rules engine.	Choose high-signal paths and expect to investigate/accept legitimate changes.
Live response	Bounded named commands; no arbitrary remote shell.	Safer capability surface, with less general remote administration power.
Platform reach	Packaged deployment and strongest endpoint path are Linux-oriented; some policy logic understands macOS/Windows reports.	Validate collection and response behavior on every supported endpoint class.
Console transport	Port 47261 is HTTP by default.	Use a trusted network or reverse-proxied HTTPS before carrying admin tokens across shared networks.
Multi-tenancy	Admin/viewer roles, not organization/team isolation.	Deploy separate instances when strong tenant boundaries are required.

19. Reference

Ports

Port	Service	Recommended reachability
47261/tcp	Console and agent downloads (HTTP)	Administrative browsers and agents; preferably behind HTTPS.
47262/tcp	Enrollment and CA service (HTTPS)	Hosts being enrolled and enrolled agents as required.
47263/tcp	Agent control (mTLS gRPC)	Enrolled agents.
47264/tcp	Administrative API	Loopback only.
5432/tcp	PostgreSQL	Loopback only in the packaged deployment.

Important commands

```
# Retrieve admin token from Proxmox host
pct exec <CTID> -- cat /var/lib/defendsec/admin-token.txt

# Enter the container
pct enter <CTID>

# Server health and logs
systemctl status defendsec-apid defendsec-console
journalctl -u defendsec-apid -u defendsec-console -f

# Agent health and logs
systemctl status defendsec-agentd
journalctl -u defendsec-agentd -f

# Update diagnostics
systemctl status defendsec-update.service
journalctl -u defendsec-update.service --no-pager -n 100
```

Glossary

Term	Meaning in DefendSec
Agent	Endpoint process that collects inventory and, for the Go implementation, receives signed commands.
Alert	Durable, triageable security observation with status and provenance.
Advisory	Catalog entry describing a vulnerable-version floor and severity.
Baseline	Accepted SHA-256 state for watched paths on a host.
FIM	File Integrity Monitoring; detecting changes to watched files.
mTLS	Mutual TLS, where both server and client present cryptographic identity.
PKI	Public Key Infrastructure used to issue and validate endpoint certificates.
SCA	Security Configuration Assessment; a defined host check that reports pass/fail/detail.
Viewer	Read-only console role that cannot enroll, mutate findings, respond, revoke, or update.

Source and further reading

- Repository: github.com/WASP512/defendsec

- Installation guide: docs/INSTALL.md
- Operations guide: docs/OPERATIONS.md
- Protocol definition: proto/defendsec/v1/agent.proto

DefendSec

Self-hosted security inventory

This document describes the codebase as of September 2026. Review release notes for subsequent changes.